Autonomous Greenhouse Strawberry Harvesting with a KUKA YouBot: A Measured Status Report on What Works, What Does Not, and What Comes Next

Franck Armand Josias Einstein
Projet de Fin d'Année — Smart Agriculture / Digital Twin
ENSA
Email: josiasarmand40@gmail.com

***Abstract*—This report gives the measured state of an autonomous mobile-manipulation stack for greenhouse strawberry harvesting, built on a KUKA YouBot and developed in two phases: a Webots prototype and a ROS 2 Jazzy / Gazebo Harmonic digital twin of a 9.90×4.90 m greenhouse. Unlike a design document, it separates three things that are usually blurred: what has been measured to work, what has been measured not to work, and what is planned. Every quantitative claim is taken from an instrumented run and is reproducible from the repository. The navigation and mapping layer is mature: the greenhouse is reconstructed to -4 cm / $+6$ cm in interior dimensions with 100% coverage and 0.0% phantom clutter, in 81 s per survey lap at 0.50 m/s with 1% of time spent under protective stop. The harvesting layer is not: the colour-threshold fruit detector reaches only 69% recall with roughly two of every three map estimates spurious, and the harvest phase spends 39% of its time in visual alignment attempts that end without a pick. Three negative results are reported in full, including a homemade correlative scan-matching SLAM front-end measured to be worse than the odometry prior it was meant to correct. Three defects and their root causes are documented, of which one — a path follower that translated and rotated simultaneously — accounted for a $46\% \rightarrow 1\%$ reduction in blocked time when corrected. The report closes with a prioritised improvement plan covering perception, control, state estimation and a ten-stage hardware commissioning procedure, and states plainly the four assumptions that will break on first contact with a real greenhouse.**

***Index Terms*—Agricultural robotics, autonomous mobile manipulation, digital twin, ROS 2, Gazebo, mecanum drive, occupancy grid mapping, model predictive control, state feedback, sim-to-real transfer, commissioning.**

I. Introduction and Purpose of This Report

The project builds an autonomous robot that surveys a greenhouse, localises ripe strawberries and picks them, targeting eventual deployment on real hardware in a real greenhouse. Development proceeded in two phases: Phase A, a self-contained Webots R2025a prototype, and Phase B, a ROS 2 Jazzy stack running against a Gazebo Harmonic digital twin.

This document is not a design description; a separate, unbounded technical report covers the design. This one answers a narrower question that matters more at this point in the schedule: *given what has actually been measured, what is finished, what is not, and what is the plan?*

Three commitments shape it.

Every number comes from an instrumented run. The stack carries three measurement nodes that score the robot against Gazebo ground truth on every run — `truth_monitor` (pose and fruit position), `map_eval` (reconstructed map against the world description) and `perf_monitor` (time and throughput). Numbers in bold are logged measurements. Numbers that are not bold are estimates or derivations, and are labelled as such.

Negative results are reported, not omitted. Three subsystems were built, measured and found wanting. They appear here with their measurements, because a subsystem that was tried and failed is a stronger contribution than one that was quietly dropped.

The distinction between "works in simulation" and "works" is kept explicit. Section VI lists the four assumptions the simulation makes that a real greenhouse violates, with the measured or derived magnitude of each.

II. System Under Evaluation

A. The twin and the robot

The Gazebo world is a 9.90×4.90 m greenhouse: glass walls, three raised growing gutters 0.80 m high at $y = -1.2, 0, +1.2$, each 8.0 m long and 0.40 m wide, leaving four driving aisles 0.80 m wide plus open cross-corridors at both ends. It contains 127 individually placed berries at known positions, which is what makes fruit localisation scoreable rather than merely demonstrable.

The robot is a KUKA YouBot: a 0.58×0.38 m mecanum base [1] carrying a 2D lidar at base centre (360 bearings over 360 °), an RGB camera on a mast, and a five-degree-of-freedom arm. The base is driven kinematically by Gazebo's `VelocityControl` and has no collision geometry, so containment is entirely a software responsibility — a fact that will recur in Section V.

B. Software architecture

Table I lists the running nodes. Two architectural rules govern the whole stack and both were adopted after a failure.

Table I: Running nodes in Phase B

Node	Responsibility
`mapping_node`	log-odds occupancy grid from lidar; publishes raw and inflated
`planning_node`	A* on the inflated grid, octile heuristic
`navigation_node`	pure-pursuit following, stuck detection, escape
`safety_node`	the single protective stop; the only writer of `/cmd_vel`
`mission_node`	survey → harvest state machine, alignment, picking
`camera_pan_node`	owns the ±90° camera head; publishes the measured angle and a settled flag
`strawberry_detector`	colour segmentation, ground back-projection
`arm_node`	five-joint pick sequence
`truth_monitor`	pose and fruit error against Gazebo truth
`map_eval`	reconstructed map against the world description
`perf_monitor`	time budget and throughput

Single ownership of the velocity command. Every planner writes `/cmd_vel_raw`; only `safety_node` writes `/cmd_vel`. Two guards that disagreed about what "blocked" meant once pinned the robot against a wall for an entire run.

One place for every physical number. All thresholds live in one parameter file, and `scripts/check_regressions.py` (**242** automated checks) refuses to start the stack when the file and the node defaults disagree. This check was added after a parameter drift silently doubled every clearance threshold and cost a full run.

C. Method of measurement

A reference run consists of three survey laps of a boustrophedon covering all four aisles, followed by a harvesting phase on a patrol route. The measurement nodes report at each lap boundary, periodically, and once at shutdown, so an interrupted run still yields a report. All timings below are in *simulated* time; the observed real-time factor was **0.47**, so wall-clock durations are roughly twice the simulated ones.

D. Reproducibility

Every measurement in this report is reproducible from the repository with the commands below on Ubuntu 24.04 with ROS 2 Jazzy and Gazebo Harmonic (gz-sim 8.11):

```
colcon build --symlink-install
source install/setup.bash
python3 scripts/check_regressions.py    # 0
242 pre-flight checks
ros2 launch youbot_bringup sim.launch.py
```

The pre-flight script is not optional decoration: it refuses to launch when the parameter file and the node defaults disagree, which is the failure that invalidated an earlier run entirely (Section V). The three measurement nodes start with the stack and print their reports to the same console, so a run's evidence is a single log file.

Table II: Map quality after one survey lap (source `/map_raw`)

Quantity	Measured	True
Interior length	**9.86 m**	9.90 m
Interior width	**4.96 m**	4.90 m
Dimensional error	**−4 cm / +6 cm**	
Floor coverage	**100%**	—
Phantom clutter	**0.0%** (0 cells)	—
Growing rows found	**3 of 3**	3

Table III: Time budget of a complete reference run (simulated time)

Phase	Driving	Blocked	Working
Survey lap 1	**92%**	**1%**	**6%**
Survey, laps 1–3	**93%**	**1%**	**6%**
Harvest phase	**52%**	**10%**	**39%**
Whole mission	**69%**	**6%**	**25%**

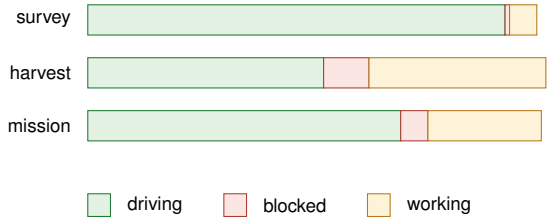

Figure 1: Where the mission time goes. The survey phase is close to saturated: **92%** of it is productive motion. The harvest phase is not, and the **39%** amber band is the single largest recoverable loss in the system — it is alignment attempts that end without a pick, not picking.

III. What Works

A. Mapping and coverage

The reconstruction of the greenhouse is the strongest result in the project. Table II gives the map evaluation after the first survey lap.

Two points deserve emphasis. First, this quality is reached after *one* lap: laps two and three, costing 163 s of simulated time between them, produced no measurable improvement. That is an actionable finding and appears in the plan (Section VII). Second, the map is built on wheel odometry alone, with no SLAM correction, which sets the context for the negative result of Section IV-B.

B. Navigation throughput

Table III gives the time budget. The `perf_monitor` node classifies every control tick into one of three buckets: *driving* (a translation command is being executed), *blocked* (a translation was commanded and the protective stop cancelled it) and *working* (stopped on purpose — aligning on a fruit or running an arm cycle).

Survey laps took **81 s**, **81 s** and **82 s** — a spread of 1 s over three laps, which is the repeatability one wants from a deterministic coverage route. Mean speed over the whole

mission was **0.50 m/s** against a cruise setting of 0.60 m/s, with **40.9 m** covered in the first lap.

C. Protective stop

The guard measures clearance from the *footprint edge* along the commanded direction of travel, not from the lidar at the base centre, using a rectangular footprint model (half-length 0.29 m, half-width 0.19 m) rather than a circumscribed disc. It is also self-explaining: on every intervention it reports what stopped it, in both body and world coordinates. A representative log line:

```
Translation held: heading +0 deg (body), obstacle
at body (+0.39, +0.24), 0.10 m from the bumper
 = world (+3.15, +0.01).
```

This is not cosmetic. Two earlier attempts to fix the guard made it worse because the log said only "obstacle within 0.12 m", which in a 1.05 m lane is a puzzle rather than a diagnosis.

D. Manipulation

Multi-pick per station works: the robot harvests everything within reach before moving on, as commercial harvesters do. The reference run logged `Pick done (3 at this stop)` and `Pick done (2 at this stop)`, for **16** berries at **2.7** per simulated minute, or **22 s** of simulated time each.

E. What transferred from Phase A unchanged

A secondary result, and one that justifies the two-phase structure: the algorithm layer developed in Webots ported to ROS 2 without redesign. Mecanum inverse kinematics with direction-preserving saturation, log-odds occupancy mapping with exact Bresenham ray casting, A* with the octile heuristic on an inflated grid, and pure-pursuit following all moved across as framework-independent libraries, with only their tuning constants re-derived for the narrower greenhouse. Three constants changed and each change was traceable to the environment rather than the algorithm: the lookahead distance fell from 0.40 m to 0.32 m, the inflation radius from 0.32 m to 0.18 m, and the footprint model changed from a circumscribed disc to a rectangle.

That last change is the interesting one. A disc of radius $\sqrt{0.29^2 + 0.19^2} = 0.347$ m is correct for a body that may be arbitrarily rotated, and it is what Phase A used in 1.2 m aisles. In 0.80 m aisles it is 16 cm too conservative sideways, which is the difference between a passable lane and an impassable one. The rectangular model, evaluated along the commanded direction as $\min(a/|\cos\theta|,\, b/|\sin\theta|)$, restores the margin.

F. Engineering infrastructure

Table IV lists what exists around the robot code. This is reported as a result because it is what makes every other number in this document trustworthy.

Table IV: Verification and measurement infrastructure

Item	Size
Automated pre-flight regression checks	**242**
Parameter file / node default cross-checks	**24 parameters**
Ground-truth scoring nodes	3
Single-subsystem test controllers (Phase A)	11
Hardware commissioning stages (new)	10

Table V: Fruit detection and localisation, measured against ground truth

Quantity	Measured
Recall (real berries reported)	**≈69%**
Map estimates matching no real berry	**66 of 111**
Position error of matched estimates	**0.21 m**

IV. What Does Not Work

A. Fruit detection — the dominant bottleneck

The detector thresholds redness by channel ratio ($G < 0.33R$ and $B < 0.33R$), groups the mask into blobs, and back-projects each centroid onto the known plant-top plane. Table V gives its measured performance against the 127 known berry positions.

Roughly **three of every five** map estimates are spurious. The consequence is visible one row up, in Fig. 1: the robot stops beside things that are not fruit, creeps toward them, fails to centre anything, and times out. Log evidence is abundant — `Alignment stalled ...resuming patrol`, `Alignment lost/timed out` — and it accounts for most of the **39%** amber band.

There is a structural reason the detector cannot be repaired by tuning: **no fruit map exists**. The detector publishes per-frame detections and nothing on the robot accumulates them. The 111 estimates counted above are tallied by the external scorer, not by the robot. A berry seen once, in one frame, is therefore accepted as real, with no possibility of corroboration.

B. Homemade SLAM — a negative result

A correlative scan matcher [2] with a likelihood-field score [3] was implemented as an online SLAM front-end, with a coarse-then-fine search and a strictly-better acceptance gate. Measured against ground truth, **it is worse than the odometry prior it was meant to correct**. The calibrated odometry holds **0.20 m** of position error over a ten-minute mission; the scan-matched estimate does not improve on that and degrades it in the long, geometrically repetitive aisles, where the score surface has a broad plateau and shifting the scan along the aisle explains the map as well as the true pose does.

This is reported as a result rather than deleted. It bounds what a naive front-end can do in a structured, self-similar environment, and it is the measured justification for the localisation plan of Section VII-D.

C. Map degradation during harvest

Map quality is **0.0%** clutter after the survey but degrades to **4.2%** clutter by the end of the harvest phase, with interior

Table VI: Subsystem status at the reference run

Subsystem	Status	Limiting measurement
Simulation & bridge	WORKS	starts reliably
Odometry (calibrated)	WORKS	**0.20 m** / 10 min
Occupancy mapping	WORKS	$-4/+6$ **cm**, **0.0%** clutter
Path planning & following	WORKS	**0.50 m/s**, **1%** blocked
Protective stop	PARTIAL	3 conservative stops at gutter corners
Harvest patrol	PARTIAL	**39%** in failed alignments
Arm / multi-pick	WORKS	**16** berries, **22 s** each
Fruit detection	BROKEN	**69%** recall, **66/111** spurious
Fruit map (persistent)	BROKEN	does not exist
Homemade SLAM	BROKEN	worse than the prior
End-of-goal recovery	PARTIAL	75 s sim-time watchdog latency

dimensional error growing to -9 **cm**. The degradation tracks odometry drift: scans integrated from a progressively wrong pose smear obstacle edges. The map is therefore accurate when it is built and slowly becomes less so, which is the expected signature of unbounded dead reckoning and another argument for the same fix.

D. Recovery latency at the end of a goal

In the reference run the robot sat at $(+3.51, +0.75)$ for approximately 80 s of wall time, repeating `goal explore (+3.80, +0.60) | waypoint 1/4`, while `navigation_node` had already reported `Goal reached`. The distance to the goal was 0.33 m against an `ARRIVAL_TOLERANCE` of 0.30 m: the follower considered the path complete and the mission did not.

The condition is recoverable — a global watchdog fires after 75 s of *simulated* time, which at the observed real-time factor is about 160 s of wall time — but the run was interrupted before it triggered. The defect is therefore one of recovery *latency*, not of deadlock, and it is ranked accordingly in the plan.

E. Residual protective-stop conservatism

Three protective interventions in the reference run all named the same geometry: the centre gutter faces at $y = \pm 0.20$ entering the 0.24 m half-width test corridor while the robot drove the $y \approx -0.45$ aisle. The guard is not wrong; it is slightly wider than a 0.80 m aisle can afford.

F. Summary

V. Defects Diagnosed and Corrected

Three defects were found, root-caused and fixed during this reporting period. They are documented because the root causes generalise.

A. Crabbing: the follower translated while rotating

The pure-pursuit follower resolved the world-frame velocity toward the lookahead point into the body frame and issued it *simultaneously* with a proportional yaw command. This is legal for a holonomic base and it is what Phase A used successfully in 1.2 m aisles. In 0.80 m aisles it has two consequences: the swept width of a 0.58×0.38 m body crossing at 45° grows from 0.38 m to 0.68 m; and the protective stop, which tests the corridor swept along the *commanded* direction, aims its test rectangle into the gutter beside the robot rather than down the lane ahead.

Table VII: Effect of the heading-first correction

Quantity	Before	After
Time blocked	**46%**	**1%**
Survey lap duration	**295 s**	**82 s**
Mean speed	**0.17 m/s**	**0.50 m/s**
Berries harvested	**5**	**16**

The correction is to face the target first and translate along the body x axis scaled by $\cos(e_\psi)$, reserving holonomic translation for the two manoeuvres that genuinely need it. Table VII gives the measured effect.

Generalisable lesson: a design can be correct in one environment and wrong in another for reasons no code review surfaces. Only a narrower corridor and a time measurement exposed this.

B. Clearance measured from the wrong origin

The protective stop compared lidar ranges, measured from the sensor at the *base centre*, against a 0.28 m threshold — while the body's half-length is 0.29 m. The brake therefore fired when the wall was 1 cm *inside* the robot's own nose. Because the base has no collision geometry, nothing in the physics stopped it either. The fix was a shared clearance library, used by both the guard and the recovery so they cannot disagree, in which every distance is measured from the footprint edge along the direction of travel.

C. Parameter drift between file and code

After the clearance semantics changed, the parameter file retained the old centre-relative values. The file wins at runtime, so every threshold silently doubled: 0.28 m from the bumper is 0.57 m from the centre. That run spent **58%** of its time blocked and harvested **5** berries instead of **27**. The fix was not vigilance but automation: a start-up check that cross-references all **24** parameters against the node defaults and refuses to launch on disagreement. It immediately caught three further silent drifts.

VI. Sim-to-Real Gap

Four assumptions hold in the twin and fail in a greenhouse. They are listed with magnitudes so they can be planned against rather than discovered.

1. Glass is opaque to the lidar. In Gazebo, walls return every beam. A real 2D lidar at glancing incidence on glass receives a weak return or none, so the wall becomes invisible. This is the most dangerous of the four, and its mitigation is physical: an opaque band at lidar height along every reachable pane.

2. The floor does not slip. The base is driven kinematically, so mecanum roller slip is unmodelled. On a wet greenhouse floor, slip corrupts heading worst, and heading error propagates directly into fruit position: a 5° heading error at 2 m displaces

Table VIII: Prioritised improvement plan

#	Action	Targets	Effort
1	Mission restructure: 1 survey lap, 2 scouting laps, then harvest	163 s of wasted survey; no fruit map	low
2	Persistent fruit map with two-pass confirmation	**66/111** spurious	low
3	Arrival-tolerance reconciliation	75 s recovery latency	low
4	Narrow the corridor half-width	3 conservative stops	low
5	Synthetic dataset with domain randomisation	no training data	medium
6	Learned detector, evaluated offline	**69%** recall	medium
7	State feedback on the alignment loop	**39%** working time	medium
8	IMU + EKF + AMCL localisation	**0.20 m** drift; map decay	medium
9	Pan head, $\pm 90^\circ$	coverage per lap	medium
10	MPC as a comparison to pure pursuit	constraint handling	high

a back-projected estimate by 0.17 m. Measured drift of 0.20 m per ten minutes in simulation is not a prediction for hardware.

3. Illumination is constant and neutral. The colour-ratio test is invariant to illumination *scaling*, which is why it survives shade, but not to colour *temperature*. Late-afternoon light through glass lowers G/R for the entire scene, at which point everything passes the fruit test.

4. Stopping is instantaneous. The protective distance of 0.12 m is correct for a kinematic model. For hardware, ISO 13855 gives

$$S = K\,(t_{\text{react}} + t_{\text{stop}}) + C, \tag{1}$$

and with a scan age plus control period plus drive latency of $t_{\text{react}} \approx 0.20$ s, a mechanical stop of $t_{\text{stop}} \approx 0.30$ s and a margin $C = 0.20$ m at $K = 0.30$ m/s, the required distance is 0.35 m — close to three times the simulation value. This is arithmetic, not caution.

VII. Improvement Plan

Table VIII ranks the planned work by measured benefit against effort. The ordering is deliberate: items are sequenced so that each one is verifiable by an existing measurement before the next begins.

A. Mission restructure and the fruit map

Map quality saturates after lap one (Section III), so two of the three survey laps are pure cost. They are replaced by two *scouting* laps, during which the robot detects but does not pick, and a persistent fruit map accumulates observations, merging detections within a fusion radius as the crate discovery of Phase A already does.

The value is not the time saved. It is that two spatially distinct passes provide a **corroboration criterion**: a candidate seen from only one pass is rejected. This is the direct structural attack on the **66 of 111** spurious estimates and, through them, on the **39%** of harvest time spent aligning on nothing. The target mission becomes `SURVEY` $\times 1 \rightarrow$ `SCOUT` $\times 2 \rightarrow$ `HARVEST`.

B. Perception: data first, model second

The site is not accessible for photography, so the training set will be generated from the digital twin with domain randomisation over the nuisance variables that break the current detector: colour temperature and intensity, sun direction and shadows, ripeness as a continuum rather than a binary, occlusion by foliage, and deliberately included red distractors. The simulator supplies the annotations, which is the one thing a twin gives that a photographer does not.

The limitation is stated plainly: a model trained only on rendered images will transfer imperfectly. The plan is synthetic data plus public strawberry datasets [4] now, with fine-tuning on a few hundred real annotated images once the site is reachable. Evaluation is offline, against human annotations, and **against the current colour threshold on the same images** — otherwise the comparison proves nothing.

C. Control: state feedback where it is measurable

The visual alignment loop is currently proportional with a dead zone and a saturation, and its logged failure signature is a steady-state error: an offset that sits at $+0.04$ to $+0.08$ and never centres. That is precisely the condition a proportional law with a dead zone cannot remove.

The loop is recast in discrete state-space form. With δ the lateral offset of the target in the image and v_y the commanded lateral velocity,

$$X(k) = \begin{pmatrix} \delta(k) \\ \sum \delta \end{pmatrix}, \qquad X(k{+}1) = A\,X(k) + B\,e(k), \tag{2}$$

under the state-feedback law

$$e(k) = -R\,X(k) + Q\,C, \tag{3}$$

with R obtained by pole placement for a settling time of about 1.5 s without overshoot, and Q chosen for unity static gain so the steady-state error vanishes. The image gain depends on range, which the fixed proportional gain ignores; the model makes that explicit.

Model predictive control is planned as a *comparison* rather than a replacement. Its attraction is that three things currently handled separately — velocity and acceleration limits, obstacle clearance, and the penalty on lateral velocity that suppresses crabbing — become constraints and cost terms of one optimisation over a horizon H. Its cost is a quadratic-program solver at 20 Hz, error-frame linearisation, and infeasibility handling, against an incumbent that already achieves **0.50 m/s** with **1%** blocked time. It is therefore a chapter and a benchmark, not a substitution.

D. State estimation

The homemade SLAM front-end failed; the response is not to tune it but to change the architecture, in ascending order of cost:

1) An **IMU**. Mecanum slip corrupts heading worst, and heading is what corrupts fruit localisation. A gyroscope bounds it at negligible cost, and is the highest benefit-to-cost item in this section.
2) **EKF fusion** of wheel odometry and IMU. This is the stochastic form of the Luenberger observer, $\hat{X}(k{+}1) = A\hat{X}(k) + Be(k) + L(y(k) - C\hat{X}(k))$, with L obtained from noise covariances rather than placed by hand.
3) **AMCL** against the map built in a dedicated mapping run. This supplies the *absolute* correction the scan matcher failed to provide, and turns drift from cumulative into bounded.
4) **Surveyed fiducials** at gutter ends, giving centimetre-accurate absolute fixes and global localisation from an unknown start pose.

E. Sensing geometry

A pan head of $\pm 90°$ is being added, decoupling where the robot looks from where it drives, so both plant rows are observable from one aisle. Two fixed cameras would achieve the same with no moving part and are mechanically preferable in a humid environment; the pan head was chosen on cost. The trade-off is recorded explicitly: the pan angle enters the fruit back-projection in series with the robot yaw, so images must be taken with the head settled, never mid-sweep.

F. Hardware commissioning

A ten-stage commissioning package has been built, each stage a runnable node with a written procedure, a numeric pass/fail criterion and an archived JSON and Markdown report. Stages run in order and a stage that has not passed blocks those after it. The sequence is: emergency stop and speed bridle; wheel command; UMBmark odometry calibration [5] on the real floor; lidar validation against real glass; manual mapping; bounded localisation over thirty minutes; autonomous navigation in an empty greenhouse; dataset collection; offline detector evaluation; and supervised single-fruit picking.

Two stages are expected to fail first, and both change the plan rather than the tuning: lidar-versus-glass, whose remedy is to modify the greenhouse, and the thirty-minute localisation bound, whose remedy is the estimator stack above. Every physical constant lives in one profile file with `sim` and `hardware` variants, so the protective distance moves from 0.12 m to the 0.35 m of (1) by selecting a profile rather than by editing code.

G. Sequencing

Table IX groups the plan into three blocks, ordered so that each block is verifiable by instruments that already exist before the next begins. The first block requires no new hardware, no new data and no new theory, and it addresses the two largest measured losses in the system.

Block A is the only one whose benefit is already quantified by an existing measurement, which is why it is first: the **39%** of harvest time and the **66 of 111** spurious estimates are both

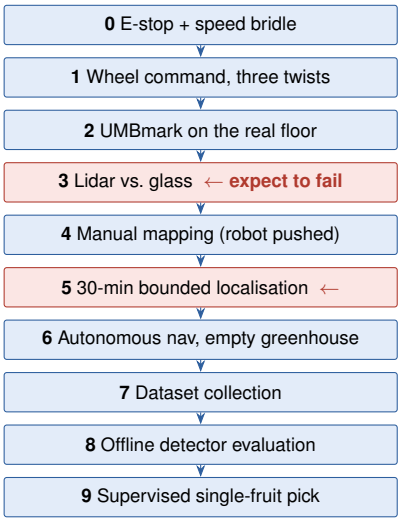

Figure 2: The ten commissioning stages. Each removes exactly one unknown, and a stage that has not passed blocks every stage after it. The two highlighted stages are the ones expected to fail first; in both cases the remedy changes the plan — modify the greenhouse, or change the estimator architecture — rather than the tuning.

Table IX: Work sequencing and the measurement that closes each block

Block	Contents	Closed when
A — software only	items 1–4: mission restructure, fruit map, arrival tolerance, corridor width	harvest *working* time below 20%; spurious estimates below one in five
B — perception	items 5–7: synthetic dataset, learned detector, alignment state feedback	stage-8 precision and recall above target and above the colour baseline
C — hardware readiness	items 8–10 plus commissioning stages 0–9	stage 5 shows a flat drift slope over 30 min

instrumented today, so the effect of the change will be visible on the next run without building anything new to see it.

Finally, and separately from all of the above: the speed clamp and arming interface in that package are *functional* safety only. They are Python in a non-real-time process. Protection of people requires a hardware emergency stop that cuts motor power without the computer's participation, a bridled speed, and — for operation alongside workers — a certified safety scanner independent of ROS [6].

VIII. Threats to the Validity of These Measurements

The numbers above are only as good as the way they were obtained, and four limitations should be stated by the author rather than found by the reader.

Single-run reporting. The reference run is one run. Survey lap times were repeatable to 1 s across three laps, which supports the navigation figures, but the harvest figures — **16**

berries, **39%** working time — come from one harvest phase and carry no variance estimate. A statistically defensible claim needs the run repeated with different random seeds, which is planned alongside block A.

The scorer shares a world model with the robot. `map_eval` compares the reconstructed map against the same world description the simulator was built from, so it validates the mapper but cannot detect an error in the world description itself. The same holds for fruit positions. This is unavoidable in simulation and disappears on hardware, where the tape measure is genuinely independent.

Recall is measured against berries in the world, not berries that were visible. The **69%** figure counts all 127 berries, including those the camera never had a viewpoint on. It therefore conflates detector recall with route coverage and understates the detector. Separating the two requires per-frame visibility ground truth, which the offline evaluation of stage 8 will provide.

Sim time versus wall time. All durations are simulated. At a real-time factor of **0.47**, a mission reported as 9 min 54 s took 21 min 13 s on the wall. Timeouts specified in simulated time behave differently in wall-clock terms, which is exactly what produced the apparent deadlock of Section IV: a 75 s watchdog is a 160 s wait for the person watching.

IX. Conclusion

The navigation half of this system is finished to a standard that can be defended with measurements: a greenhouse reconstructed to -4 **cm** / $+6$ **cm** with **100%** coverage and **0.0%** clutter, surveyed in **81 s** per lap at **0.50 m/s** with **1%** of time under protective stop, and a harvesting arm that picks multiple fruits per station.

The perception half is not, and the bottleneck is identified rather than suspected: a colour-threshold detector at **69%** recall with **66 of 111** map estimates spurious, feeding a mission that has no persistent fruit map and therefore no way to disbelieve a single observation. That one structural gap explains the **39%** of harvest time spent in alignments that end without a pick, and it is the first item of the plan.

Three subsystems were measured and found wanting, and are reported as such. Three defects were root-caused and corrected, one of which — a path follower that translated while rotating — accounted for a $46\% \rightarrow 1\%$ reduction in blocked time. The remaining work is ordered so that each step is verifiable by an instrument that already exists, and hardware deployment is gated behind ten commissioning stages whose criteria are written before the robot is switched on rather than after.

References

[1] B. E. Ilon, "Wheels for a course stable self-propelling vehicle movable in any desired direction on the ground or some other base," U.S. Patent 3 876 255, 1975.

[2] E. B. Olson, "Real-time correlative scan matching," in *Proc. IEEE Int. Conf. Robotics and Automation*, 2009, pp. 4387–4393.

[3] S. Thrun, W. Burgard, and D. Fox, *Probabilistic Robotics*. Cambridge, MA: MIT Press, 2005.

[4] I. Sa, Z. Ge, F. Dayoub, B. Upcroft, T. Perez, and C. McCool, "DeepFruits: A fruit detection system using deep neural networks," *Sensors*, vol. 16, no. 8, p. 1222, 2016.

[5] J. Borenstein and L. Feng, "Measurement and correction of systematic odometry errors in mobile robots," *IEEE Trans. Robotics and Automation*, vol. 12, no. 6, pp. 869–880, 1996.

[6] *Industrial trucks — Safety requirements and verification — Part 4: Driverless industrial trucks and their systems*, ISO 3691-4, 2020.